Confidence Layer

Optimising session quality, not conversion rate

FEG Innovation Hackathon 2026 · Challenge 1 — Session Quality and Session-to-Action Conversion

Position

Abandonment at the confirm step is conventionally read as a motivation gap and closed with urgency or incentive. We read it as an information gap. A user who has assembled a selection and reached the point of confirming is not short of motivation — they are short of certainty about what the market means, whether the price moved since they added it, or whether the slip reflects what they intended. FEG's brief asks for confident, informed actions. We propose optimising for precisely that, and replacing conversion rate as the objective in order to make it possible.

The problem, reframed

FEG name three leaks: sessions ending in browsing, discovery friction from undifferentiated layouts, and drop-off at the final step. We target the third, because it is both the sharpest and the most commonly misdiagnosed.

The standard playbook for final-step abandonment is pressure — countdown timers, scarcity copy, social proof, deposit inducements. Under FEG's guardrail every one of these is disqualifying. We also read them as ineffective for the population that matters here: a user who lacked motivation would not have reached confirm. What is missing at that moment is resolution, not persuasion.

This is our interpretation rather than a measured finding, and we flag it as such. But it is consistent with the shape of the evidence FEG describe — a product with no traffic problem that leaks at its highest-intent moment. **The interface goes quiet exactly when a question has formed.**

Session Quality Score

Our first deliverable is a metric, not a model. FEG list “better ways to measure session quality itself” as an innovation area and state that metric choices are themselves judged, which we read as an acknowledgement that conversion rate is the wrong optimisation target for a regulated product.

We propose a composite per-session score:

$$SQS = w_1 \cdot \text{action\_value} + w_2 \cdot \text{discovery\_efficiency} - w_3 \cdot \text{harm\_indicator\_load}$$

Action value captures completed actions weighted by value. Discovery efficiency captures time to first action and depth reached, rewarding sessions that get the user somewhere rather than sessions that merely last. The third term is a subtractive penalty derived from harmful-play indicators — loss-chasing velocity, session-length anomaly against the user's own baseline, deposit-frequency deviation, and time-of-day drift.

The subtraction is the design decision. Because the protection term carries a negative weight, **any decisioning policy that increases harmful-play indicators mathematically cannot increase the score.** The guardrail stops being a rule we undertake to follow and becomes a property of the objective we optimise. Sessions showing harmful-play indicators cannot rise, because rising is defined as making them fall.

One condition governs this, and we state it plainly rather than leaving it implicit. The guarantee holds only while the harm model remains architecturally independent of the conversion model. If the two ever share a training signal, a feature store, or a gradient, the optimiser can learn to suppress the indicator rather than the harm, and the property collapses. Independence is therefore a hard constraint on our architecture, not an implementation preference.

A second-order consequence is worth naming: because SQS can reward a session that correctly ends without an action, our system is able to decline to intervene and still score well. No conversion-rate optimiser can represent that outcome.

The Confidence Layer

The second deliverable is the mechanism that acts on the score. It is a thin, embeddable decisioning layer, not a platform.

Detection runs on client-side signals available at the confirm step: dwell on the confirm control, stake edits, scroll-back to the originating market, focus loss, and odds movement since the selection was added. These are captured by a consent-gated SDK that transmits no personally identifying data. A gradient-boosted classifier estimates, from the opening seconds of that moment, whether the session is heading toward abandonment.

Response is where our position becomes concrete. Where a conventional system would apply pressure, the Confidence Layer supplies information: a plain-language explanation of the selection, what has changed since it was added, what the stake actually returns, and an explicit, low-friction path to defer — keep it in the slip, decide later. The intent is that the user leaves the moment more certain, whichever way they decide.

Which response fires is chosen by a contextual bandit optimising SQS rather than click-through, learning online instead of waiting on multi-week A/B cycles.

Gate ordering matters more than model quality here, and we treat it as architecture. Self-exclusion status, at-risk flags, consent state and responsible-gambling limits are evaluated as hard binary gates *before* any candidate response is scored. A user who fails a gate produces no candidate set, so there is nothing for the optimiser to score around. **A refusal that can be outweighed by a sufficiently high predicted reward is not a refusal.**

Build plan

Our approach is to fork the plumbing and write the scoring, on the reasoning that the scoring is the part nobody has open-sourced and the part the rubric actually rewards.

For signal capture we will either fork PostHog, which provides autocapture, session replay and experimentation in a self-hostable package that supports the GDPR position, or ship a thin custom SDK feeding ClickHouse. That decision is being made before the sprint on the basis of team familiarity, since forking PostHog is a real time investment and a wrong call there costs hours we cannot recover.

For the decision loop we will build on `VowpalWabbit/learn_to_pick`, a Python contextual-bandit library whose news-recommendation example is structurally identical to our problem: select from candidate actions given context, score the decision, feed the reward back. We cite `VowpalWabbit/reinforcement_learning` — RLClientLib, the library behind Microsoft's Decision Service — as our production reference, specifically for its handling of the delayed-reward join. Our decision fires at the confirm step while the outcome settles seconds or minutes later, and naive implementations corrupt their training data at exactly this seam.

The hesitation classifier is a standard LightGBM model and is not novel. The Session Quality Score and the harm-indicator models are ours, because no open-source equivalent exists.

Evidence base

Our harm indicators derive from published research rather than intuition, which we consider necessary given that a subtractive penalty term built on invented features would make the compliance argument decorative.

The Transparency Project, run by the Division on Addiction at Cambridge Health Alliance, hosts anonymised behavioural datasets from bwin that underpin much of the loss-chasing literature. The repository `alro5921/problem-gambling-study` works with that data across roughly four thousand accounts, approximately half of which were flagged by the operator's own Responsible Gaming system — providing labelled ground truth for harm rather than a proxy. `hkalager/ML_Gambling` is the replication code for Hassanniakalager and Newall, *A machine learning perspective on responsible gambling*, published in *Behavioural Public Policy*.

Dataset access status: **requested, awaiting approval**. Access terms are being confirmed directly and we do not assume approval.

Compliance by design

We treat compliance as an architectural property, and the ordering of operations is the substance of that claim.

- **Gate ordering.** Self-exclusion, at-risk flags, consent state and responsible-gambling limits are evaluated before candidate generation. No intervention exists to be scored for a gated user. This is the difference between a policy and a guarantee.
- **Objective-function containment.** The harmful-play penalty sits inside SQS with a negative weight, so harm reduction is not a constraint applied to the optimiser but a component of what the optimiser maximises.
- **Model independence.** The harm model is trained separately, on separate features, and is never exposed to the conversion objective. Documented as a hard architectural constraint, because the guarantee above depends on it.
- **No dark patterns by construction.** Our candidate response set contains no urgency mechanic, scarcity signal, social-proof element or inducement. These are absent from the action space, not filtered out of it, so the bandit cannot discover them.
- **Generative output containment.** The plain-language explanation layer is retrieval-grounded and template-constrained. Odds, returns and market descriptions are injected from source data and cannot be generated freely, eliminating hallucinated prices. A deterministic fallback ships before the model does.
- **EU baseline.** GDPR and ePrivacy: consent-gated capture, no PII in transit, anonymised sample data only. EU AI Act: the decisioning model is documented for transparency and human oversight, with intervention logs auditable per session. EAA/WCAG 2.1 AA: interventions render as accessible text, never as motion or colour alone.

- **Auditability.** Every decision writes a record — gates evaluated, candidates generated, response selected, outcome settled. This log is simultaneously the compliance artefact and the dataset behind our impact case.

Scope honesty

Live in the 24-hour demo: the capture SDK, the hesitation classifier, the bandit loop, the SQS computation, the gate ordering, the sandbox betslip frontend, and holdout instrumentation from the first hour. Presented as production architecture rather than built: the Kafka/ClickHouse ingest path and the Kubernetes serving topology.

The likeliest failure is the generative explanation layer — model latency, template drift, or a hallucinated price. Our fallback is deterministic templated copy, built first and shipped regardless. If the model does not work by morning, the demo still runs and the argument is unaffected, because our thesis is about what information the user receives, not about how the sentence is produced.

Our impact case will quantify final-step uplift against a stated baseline. Real baselines arrive only after shortlisting and NDA, so every figure we present will carry its assumption in writing, and harm-indicator neutrality will be reported as a hard constraint rather than a favourable outcome.

Team

- **Saketh Suman Bathini — Team Lead.** Solution architecture, Session Quality Score design, demo frontend.
- **Sai Sharath Chandranand Netha Gundeti — Decisioning.** Architecture, Bandit loop, hesitation classifier, reward plumbing.
- **Hrishikesh Vuppala — Data and harm.** Harm-indicator models, synthetic session generator, holdout instrumentation.

Strongest area: **Software Engineering with expertise in Full-Stack Development, Real-Time Distributed Systems, Mobile & Backend Engineering, API Architecture, Personalization and Recommendation Systems, AI/ML Applications, and Data-Driven Product Development.**

Prior relevant work: **recommendation feature for a college project, built on collaborative filtering, An AI-powered web app using LLM APIs with retrieval grounding** .

Open questions for FEG

Three questions we cannot answer without post-NDA access, and whose answers would change our design:

- At the final step, is the dominant abandonment cause price movement, market ambiguity, or slip-composition error? Our response set is weighted differently for each.
- What harmful-play indicators does FEG already compute in production, and at what latency? We would prefer to consume existing signals than to duplicate them.
- Is the confirm step server-rendered or client-side, and what is the acceptable added latency budget before the control becomes interactive?